

Week 9. Database & Secret Management in Cloud ML Environments

- Deploy secure cloud databases and use Key Vault for managing credentials in Azure
- Learn industry best practices for responsible data access and operational security in ML workflows
- Prepare for real-world demands in AI engineering with a focus on compliance, MLOps, and automation

Learning Objectives

- Identify best practices for securing database connections in cloud-based ML workflows
- Deploy and connect to Azure managed databases from compute resources
- Store and use secrets with Azure Key Vault and environment variables; verify access control
- Build confidence in secure, automated, and auditable ML pipelines; prepare for industry assessments

Agenda

- Why secure data management matters in AI workflows
- Overview. Azure databases and Key Vault essentials
- Setting up; Deploy a managed database and connect securely
- Store and retrieve secrets; using environment variables and Key Vault
- Practical activity; Test secure connectivity and access control
- Review; Link to responsible AI and workplace standards

Why Secure Data Management?

- Data privacy and compliance are industry priorities; breaches can cause severe damage
- Machine learning often requires access to sensitive data; securing credentials and auditability is essential
- Responsible AI frameworks require robust controls for data and secrets in production environments

Azure Managed Databases

- Azure offers fully managed options like Azure SQL, Cosmos DB, and PostgreSQL; integrates with AI workflows
- Automatically handles backups, scaling, and patching; reduces operational overhead
- Secure by default; built-in network controls and encryption at rest and in transit

Use Case. ML Training on Cloud Data

- Scenario. Data science team wants to train a model using live business data
- Data is stored in Azure SQL Database; access must be restricted and auditable
- Solution. Use managed identity and Key Vault to provide controlled, automated access to the database

What is Azure Key Vault?

- Azure Key Vault is a secure service for managing sensitive information; supports secrets, encryption keys, and certificates
- Central point for credential storage; reduces the risk of secrets being leaked in code or scripts
- Granular access control; supports audit logging for compliance and investigation

Environment Variables; A Safer Way

- Best practice is never to hardcode secrets or passwords; use environment variables instead
- Scripts and ML models read credentials from the environment; keeps secrets out of source code and version control
- Combine Key Vault and environment variables for industrial-strength protection in production

Demo. Provision a Database and Store a Secret

1. Create an Azure SQL Database in the portal or CLI; note the connection string
2. Add the database connection string as a secret in Azure Key Vault
3. Use Azure CLI or Python SDK to retrieve the secret value into an environment variable
4. Connect your Python script to the database using the retrieved credentials

Example. Access Key Vault Secrets from Python

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient
import os

key_vault_url = "https://<your-keyvault-name>.vault.azure.net"
credential = DefaultAzureCredential()
client = SecretClient(vault_url=key_vault_url, credential=credential)

db_secret = client.get_secret("my-db-connection").value
os.environ["DB_CONN_STRING"] = db_secret

# Use os.environ["DB_CONN_STRING"] for your ML code
```

- Replace `<your-keyvault-name>` and `"my-db-connection"` with your setup
- This pattern enables reuse and automation; secrets are never exposed in plain code

Activity. Test Secure Connectivity

- Task; Deploy an Azure SQL Database and provision a Key Vault with a sample secret
- Retrieve the connection string using Azure CLI or a simple Python script
- Connect to the database using the secret stored in the environment; verify access is successful
- Modify permissions in Key Vault and observe the effect; discuss as a group

Practical Tips for Industry Readiness

- Always use managed identities and Key Vault integrations on Azure; avoids credential leaks
- Audit access and review logs regularly; ensures compliance and quick incident response
- Document your secret management patterns in project wikis or README files; supports team understanding and onboarding

Assessment Preparation

- Document and screenshot your process for deploying a database, creating a Key Vault secret, and connecting securely
- Demonstrate a working Python script that uses environment variables to access cloud resources
- Explain the security benefits of using managed identities and Key Vault in a short written reflection

Linking Back; From Networking to Secure Access

- Week 8 focused on networking and storage architecture; now secure your data with credential management
- Real-world ML projects build upon both secure resource access and scalable compute
- These practices support responsible AI. Critical for audits, compliance, and trust

Summary and Next Steps

- Secure database access and secret management are industry must-haves for ML workflows
- Build workflows using managed cloud services and follow best practices for MLOps
- Next week. Apply database skills in your first PyTorch portfolio project. Prepare questions and bring your setup to the lab

Further Reading and Resources

- Azure documentation; "Secure access to databases using Azure Key Vault"
- "Responsible AI" guidelines by Microsoft and industry groups
- Review sample code on Azure's official GitHub repositories for secure ML setups

